FIAP ON

DESENVOLVIMENTO JAVA ENTERPRISE

SERIALIZAÇÃO, RMI E *SOCKETS*

JOSÉ YOSHIRIRO

5

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 5.1 – Arquitetura de camadas RMI.....12

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 5.1 – Serializando 2 objetos String 6
Código-fonte 5.2 – Classe Logradouro.....7
Código-fonte 5.3 – Classe que tenta serializar uma instância de “Logradouro” 8
Código-fonte 5.4 – Classe Logradouro implementando “Serializable” 8
Código-fonte 5.5 – Classe que desserializa um texto 9
Código-fonte 5.6 – Classe que desserializa um “Logradouro” 10
Código-fonte 5.7 – Interface remota “TipoEstabelecimentoService” 13
Código-fonte 5.8 – Implementação da interface remota “TipoEstabelecimentoService”.....14
Código-fonte 5.9 – Classe que ficará “escutando” as solicitações RMI.....15
Código-fonte 5.10 – Cliente RMI para o “TipoEstabelecimentoService” 17
Código-fonte 5.11 – Saída do “TipoEstabelecimentoCliente” informando um código existente.....18
Código-fonte 5.12 – Saída do “TipoEstabelecimentoCliente” informando um código inexistente 18
Código-fonte 5.13 – Classe que usa Socket para pesquisa remota em “TipoEstabelecimentoDAO”.....20
Código-fonte 5.14 – Saída recebida pelo cliente telnet do Socket 21
Código-fonte 5.15 – Saída no cliente telnet do Socket após consultas e pedido de saída 22

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

5 SERIALIZAÇÃO, RMI E SOCKETS 5
5.1 Introdução 5
5.2 Serialização.....5
5.2.1 Serializando uma String 5
5.2.2 Só objetos *sSerializables* e primitivos podem ser serializados 7
5.3 Desserialização 9
5.3.1 Desserialização de um texto 9
5.3.2 Desserialização de um objeto do tipo logradouro 10
5.4 RMI (Remote Method Invocation).....11
5.4.1 Introdução 11
5.4.2 Implementando um Servidor RMI 12
5.4.3 Criando um Servidor RMI que realize uma consulta no TipoEstabelecimentoDAO 13
5.4.4 Implementando um Cliente RMI 16
5.4.5 Fazendo a comunicação entre Servidor e Cliente RMI 18
5.5 Sockets.....19
5.5.1 Socket Java que usa um DAO 19
5.5.2 Acessando o Socket via telnet 21
5.5.3 Por que usar Socket, se posso usar RMI? 22
REFERÊNCIAS.....23

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 SERIALIZAÇÃO, RMI E SOCKETS

5.1 Introdução

Imagine que você necessite de um sistema que “lembre” de algumas informações mesmo após ter sido encerrado e, então, executado novamente. Que recurso você usaria para essa tarefa? Imagino que pensou em banco de dados, afinal, estudamos ORM com JPA/Hibernate recentemente. De fato, essa seria uma opção, mas **nem sempre banco de dados é a melhor solução quando precisamos guardar e recuperar dados**.

Alguns exemplos: se você precisa guardar poucas informações, como as “preferências” (configurações) de um sistema; se você precisa de alguns dados dos usuários do sistema; se você precisa enviar dados de seu sistema feito em Java para outro também feito em Java. Para situações como essas, a plataforma Java oferece o recurso de **serialização** e **desserialização** de objetos, que estudaremos a seguir.

5.2 Serialização

Quando temos situações em que precisamos guardar informações de um sistema e os bancos de dados não são a melhor opção ou nem sequer são cogitados, podemos usar o recurso de **serialização** de objetos do Java. Ele permite guardar ou enviar por uma rede objetos criados num programa Java, de tal forma que qualquer outro programa também escrito em Java consiga recuperá-los. Esse é um recurso **nativo** da linguagem Java, ou seja, não é necessário importar nenhuma biblioteca no projeto.

5.2.1 Serializando uma String

No nosso primeiro exemplo de serialização, vamos serializar um texto simples, ou seja, um objeto do tipo String do Java. Veja no Código-fonte 5.1 como isso pode ser feito.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
// pacote

import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

public class SerializadorStrings
{
    public static void main( String[] args ) throws
IOException
    {
      FileOutputStream fileStream = new
FileOutputStream("texto.ser");

      String texto = "Não sou dono do mundo, mas sou filho do
dono!";

      ObjectOutputStream os = new
ObjectOutputStream(fileStream);
      os.writeObject(texto);

      os.close();
      fileStream.close();
    }
}
```

Código-fonte 5.1 – Serializando 2 objetos String
Fonte: Elaborado pelo autor (2018)

No Código-fonte “Serializando 2 objetos String”, nós criamos um método ***main()*** apenas para poder executar a classe e já verificar o resultado. Notou que indicamos na assinatura do método que ele pode lançar uma exceção do tipo **IOException**? Isso foi necessário porque, quando usamos os recursos de serialização do Java, muitos de seus métodos e construtores possuem a indicação dessa exceção que é do tipo **checada**, ou seja, obriga a usar blocos ***try-catch-finally*** ou repassar o tratamento anunciado à exceção no método criado, que foi o que fizemos.

Note que importarmos duas classes, todas do pacote **java.io**, da biblioteca padrão do Java. São elas **FileOutputStream** e **ObjectOutputStream**. A **FileOutputStream** é usada para criar um objeto utilizado para indicar o arquivo no qual um ou mais objetos serão serializados e, no exemplo, aponta o arquivo “*texto.ser*”. A **ObjectOutputStream** é usada para criar um objeto que será utilizado para escrever um ou mais objetos que desejamos serializar e, no exemplo, solicitamos que escrevesse um texto para serialização. Após serializarmos o texto,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

solicitamos o fechamento de seu **ObjectOutputStream** e, em seguida, do **FileOutputStream**. Ambos os fechamentos foram com o método **close()**. Depois do fechamento do **FileOutputStream** o arquivo "*texto.ser*" é criado em disco.

Após a execução dessa classe, um arquivo chamado "**texto.ser**" será criado no diretório do projeto. Se tentar abrir esse arquivo com um editor de texto, verá que é inelegível, afinal, trata-se de um arquivo binário que apenas outro programa escrito em Java é capaz de entender, fazendo a desserialização, que veremos daqui a pouco.

5.2.2 Só objetos *Serializables* e primitivos podem ser serializados

No exemplo do Código-fonte "Serializando 2 objetos String", vimos como serializar um texto. Isso foi possível porque a classe **String** do Java implementa a interface **java.io.Serializable** do pacote-padrão do Java. Imagine que precisamos serializar uma classe chamada **Logradouro**, que pode ser vista no Código-fonte "Classe Logradouro".

```
// pacote

public class Logradouro {

     private long id;
     private String logradouro;
     private int numero;
     private String complemento;

     // construtor, getters e setters
}
```

Código-fonte 5.2 – Classe Logradouro
Fonte: Elaborado pelo autor (2018)

Se criamos outra classe para serializar a Logradouro, ela seria como a do Código-fonte "Classe que tenta serializar uma instância de "Logradouro"".

```
// pacote

import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectOutputStream;

public class SerializadorLogradouro
{
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    public static void main( String[] args ) throws
IOException
    {
      FileOutputStream fileStream = new
FileOutputStream("logradouro.ser");

          Logradouro logradouro = new Logradouro(1, "Rua Ze
Locao", 12, "ap 150");

          ObjectOutputStream os = new
ObjectOutputStream(fileStream);
          os.writeObject(logradouro);

          os.close();
          fileStream.close();
    }
}
```

Código-fonte 5.3 – Classe que tenta serializar uma instância de “Logradouro”
Fonte: Elaborado pelo autor (2018)

Ao executar a classe do Código-fonte “Classe que tenta serializar uma instância de “Logradouro””, tomaremos uma exceção do tipo **java.io.NotSerializableException**. Isso porque nossa classe **Logradouro** NÃO implementa a **java.io.Serializable**. Podemos providenciar isso alterando o Código-fonte “Classe Logradouro”. Vamos apenas alterar a definição da classe, como pode ser visto no Código-fonte “Classe Logradouro implementando “Serializable””.

```
// pacote
import java.io.Serializable;

public class Logradouro implements Serializable {
     // atributos, construtor, getters e setters
}
```

Código-fonte 5.4 – Classe Logradouro implementando “Serializable”
Fonte: Elaborado pelo autor (2018)

Após essa alteração, a classe **SerializadorLogradouro** executaria sem erro algum. Isso porque, além de **Logradouro** implementar **Serializable**, seus atributos também implementam **Serializable** ou são tipos primitivos (**long** e **int**).

E se um dos atributos de **Logradouro** não for **Serializable** nem primitivo? Nesse caso, uma instância sua também não poderá ser serializada.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5.3 Desserialização

Quando fazemos o processo contrário da serialização, ou seja, quando lemos um arquivo de objetos serializados Java e convertemos esses objetos em objetos Java, estamos fazendo a **desserialização**. A seguir, veremos como desserializar o texto e a instância de Logradouro serializados anteriormente.

5.3.1 Desserialização de um texto

É possível recuperar o texto serializado no “texto.ser”, como podemos ver no Código-fonte “Classe que desserializa um texto”.

```
// pacote
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class DesserializadorStrings
{
    public static void main( String[] args ) throws
IOException, ClassNotFoundException
    {
        FileInputStream fileStream = new
FileInputStream("frases.ser");
        ObjectInputStream os = new
ObjectInputStream(fileStream);

        String textoRecuperado = (String) os.readObject();
        System.out.println("Texto recuperado: "
                        + textoRecuperado);

        os.close();
        fileStream.close();
    }
}
```

Código-fonte 5.5 – Classe que desserializa um texto
Fonte: Elaborado pelo autor (2018)

Note que aqui as classes usadas foram **FileInputStream** e **ObjectInputStream**. A primeira indica o arquivo em que acreditamos estar o arquivo com objetos serializados Java. A segunda serve para recuperar o conteúdo do objeto serializado. O método **readObject()** anuncia uma **ClassNotFoundException**, por isso essa exceção está na assinatura do método **main()**. Quando usamos esse

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

método, temos que indicar o *cast* para o tipo de objetos que acreditamos estar serializado (no caso, String).

Após desserializarmos o texto, solicitamos sua exibição na saída-padrão para confirmar que o programa funcionou. Depois que já lemos e desserializamos o que queríamos, fechamos os objetos das classes **FileInputStream** e **ObjectInputStream** para que o arquivo **"texto.ser"** seja liberado ao sistema operacional. Se não fecharmos esses objetos, o sistema operacional pode não conseguir ler ou alterar esse arquivo.

5.3.2 Desserialização de um objeto do tipo logradouro

É possível recuperar também o **Logradouro** que serializamos anteriormente. Podemos ver um exemplo disso no Código-fonte "Classe que desserializa um "Logradouro"".

```
// pacote
import java.io.FileInputStream;
import java.io.IOException;
import java.io.ObjectInputStream;

public class DesserializadorLogradouro
{
    public static void main( String[] args ) throws
IOException, ClassNotFoundException
    {
        FileInputStream fileStream = new
FileInputStream("logradouro.ser");
        ObjectInputStream os = new
ObjectInputStream(fileStream);

        Logradouro logradouroRecuperado =
                                (Logradouro) os.readObject();
        System.out.println("Logradouro recuperado: "
                    + logradouroRecuperado.getLogradouro());

        os.close();
        fileStream.close();
    }
}
```

Código-fonte 5.6 – Classe que desserializa um "Logradouro"
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Note que aqui fizemos basicamente o mesmo que foi feito no Código-fonte “Classe que desserializa um texto”. A única diferença é que fizemos o *cast* para a classe **Logradouro** ao invocar o método **readObject()**. Após desserializar o **Logradouro**, podemos invocar qualquer método dele, como fizemos no Código-fonte “Classe que desserializa um “Logradouro””.

5.4 RMI (Remote Method Invocation)

Nos dias atuais, em que a Internet está em quase todo lugar e se fala muito de **IoT** (Internet of Things, ou “Internet das Coisas”), a comunicação entre sistemas é de grande importância. Com Java, uma das formas de integrar sistemas que estão sendo executados em computadores diferentes na mesma rede é com o uso do **RMI** (Remote Method Invocation, ou “Invocação de Método Remoto”, em tradução livre).

5.4.1 Introdução

Imagine que você precisa fazer com que dois computadores (notebooks, celulares etc.) se comuniquem remotamente. Exemplos desse tipo de situação: pedir para um elevador descer quando você está prestes a entrar no edifício ou pedir para o ar-condicionado do carro ligar quando você começa a se preparar para sair do escritório. Uma das maneiras de obter isso é fazendo com que um programa escrito em Java solicite para outro programa, em outro computador, executar um determinado método. Na plataforma Java, isso pode ser feito com o uso do **RMI**.

O **RMI** do Java foi baseado em uma tecnologia anterior conhecida como **RPC** (Remote Procedure Calls, ou “Chamadas de Procedimento Remoto”, em tradução livre), criada na década de 1980. As chamadas são enviadas remotamente de um programa Java (o “*Cliente*”) para outro (o “*Servidor*”) por meio de uma rede. Um programa pode funcionar como *Cliente* e *Servidor* ao mesmo tempo, sem problemas. Nesse caso, ele está habilitado para enviar e receber chamadas remotas. A arquitetura do RMI do Java é dividida em três camadas (representadas na Figura “Arquitetura de camadas RMI”):

Stub/Skeleton contém as interfaces que os objetos da aplicação usam para interagir entre si. O **Stub** fica no processo *Cliente* e o **Skeleton,** no processo

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Servidor. Eles são os componentes que abstraem a complexidade de conexão com rede, envio, recebimento e tradução dos dados trocados via rede.

Referência remota é responsável pela criação e gerenciamento das referências para objetos remotos.

Camada de transporte implementa e abstrai o protocolo responsável pelas solicitações enviadas e recebidas entre os objetos remotos pela rede.

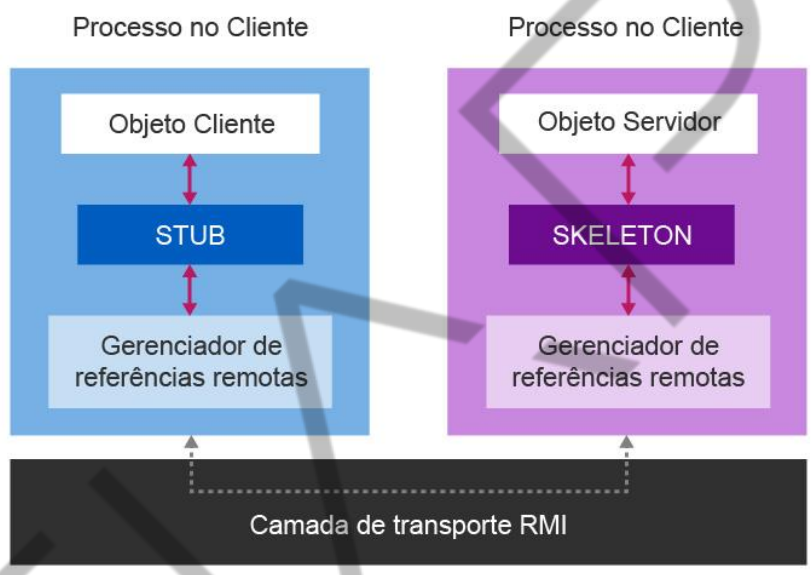

Figura 5.1 – Arquitetura de camadas RMI
Fonte: Elaborado pelo autor (2018)

5.4.2 Implementando um Servidor RMI

Um Servidor RMI é um programa Java capaz de receber pedidos para a invocação de um método enviado por um programa em outro computador. Após receber o pedido, ele executa o método solicitado e devolve o resultado dessa execução.

O primeiro passo para a criação de um Servidor RMI é criar uma **interface** que estenda **java.RMI.Remote**. O segundo passo é criar uma classe que crie e registre o ***Skeleton***, e que inicie o serviço de escuta de solicitações remotas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como exemplo de implementação de Servidor RMI, vamos criar um servidor que receba um pedido de consulta ao **TipoEstabelecimentoDAO** que vimos no capítulo **Generic DAO**. Ele vai receber um código de Tipo de Estabelecimento e devolver o nome do Tipo de Estabelecimento encontrado na busca ou **null**, se não for encontrado. Você pode colocar os códigos deste capítulo no mesmo projeto do **TipoEstabelecimentoDAO** ou criar um novo projeto e pôr o anterior como dependência, o que achar mais fácil.

5.4.3 Criando um Servidor RMI que realize uma consulta no TipoEstabelecimentoDAO

Conforme já explicado, o primeiro passo é criar a interface do serviço RMI. Vamos criar a **TipoEstabelecimentoService**, conforme consta no Código-fonte "Interface remota "TipoEstabelecimentoService"".

```
package fiapon.digital.capitulo5.rmi.server;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface TipoEstabelecimentoService extends Remote {

     public String pesquisar(Integer idPesquisa) throws
RemoteException;

}
```

Código-fonte 5.7 – Interface remota "TipoEstabelecimentoService"
Fonte: Elaborado pelo autor (2018)

Como já explicado, a *interface* que criamos estende a **java.rmi.Remote**. Isso é necessário para que possamos criar um servidor RMI. Outro detalhe é que os métodos públicos que queremos expor em nosso serviço devem anunciar uma **java.rmi.RemoteExpection** em sua assinatura. Fizemos isso no método **pesquisar()**. Vamos agora criar a implementação dessa interface, a **TipoEstabelecimentoServiceImpl**, que está exemplificada no Código-fonte "Implementação da interface remota "TipoEstabelecimentoService"". **Importante**: caso o Cliente que faça chamadas a esse Servidor seja outro projeto, apenas a **interface** deve ser disponibilizada para ele, nunca a implementação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
// pacote
import javax.persistence.EntityManager;
import javax.persistence.Persistence;
import br.com.fiap.smartcities.dao.TipoEstabelecimentoDAO;
import br.com.fiap.smartcities.domain.TipoEstabelecimento;

public   class   TipoEstabelecimentoServiceImpl   implements
TipoEstabelecimentoService {

     protected TipoEstabelecimentoServiceImpl()
     throws RemoteException {
          super();
     }

     public String pesquisar(Integer idPesquisa)
     throws RemoteException {

          EntityManager em =
          Persistence
          .createEntityManagerFactory("smartcities")
          .createEntityManager();

          TipoEstabelecimentoDAO dao =
          new TipoEstabelecimentoDAO(em);

          TipoEstabelecimento tipoEstabelecimento =
          dao.recuperar(idPesquisa);

          if (tipoEstabelecimento != null) {
               return tipoEstabelecimento.getNome();
          }
          else {
               return null;
          }
     }
```

Código-fonte 5.8 – Implementação da interface remota “TipoEstabelecimentoService”
Fonte: Elaborado pelo autor (2018)

Note que criamos um construtor na **TipoEstabelecimentoServiceImpl**. Isso é obrigatório para que seja uma implementação de serviço remoto válida. Também é obrigatório que o construtor anuncie uma **java.rmi.RemoteException**.

Dentro do método **pesquisa()**, nós iniciamos um **EntityManager** e o usamos como argumento ao criar um objeto do tipo **TipoEstabelecimentoDAO**.

Usamos o **TipoEstabelecimentoDAO** para tentar recuperar um objeto do tipo **TipoEstabelecimento**. Se for encontrado, o método retorna o seu nome. Caso contrário, o método retorna **null**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Agora precisamos criar uma classe que crie e registre o **Skeleton** e que inicie o serviço de escuta de solicitações remotas. Vamos chamá-la de **TipoEstabelecimentoServidor**. Veja um exemplo dessa classe no Código-fonte "Classe que ficará "escutando" as solicitações RMI":

```
// pacote
import java.rmi.AlreadyBoundException;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;

public class TipoEstabelecimentoServidor {

     public static void main(String[] args)  {

       try {

          TipoEstabelecimentoService service =
          new TipoEstabelecimentoServiceImpl();

          TipoEstabelecimentoService skeleton =
          (TipoEstabelecimentoService)
          UnicastRemoteObject.exportObject(service, 0);

          Registry registry =
          LocateRegistry.createRegistry(7777);

          registry.bind("TipoEstabelecimentoService",
skeleton);

          System.out.println("TipoEstabelecimentoService
registrado e pronto para aceitar solicitações.");

          }
          catch (RemoteException | AlreadyBoundException e) {

               System.out.println(
               "Não foi possível iniciar o Servidor RMI!");
               e.printStackTrace();
          }

     }
}
```

Código-fonte 5.9 – Classe que ficará "escutando" as solicitações RMI
Fonte: Elaborado pelo autor (2018)

Criamos um método **main()** na classe para que ela possa ser executada. Nele, instanciamos um objeto do tipo **TipoEstabelecimentoService** com a classe

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

que o implementa (**TipoEstabelecimentoServiceImpl**). Logo em seguida, pedimos que o Java gere seu **Skeleton** por meio do método **exportObject()** da classe **java.rmi.server.UnicastRemoteObject**. O valor **0 (zero)** que usamos nesse método indica que a porta TCP será utilizada e configurada via **registro** (*bind*) de serviço.

A seguir, começamos o processo de registro do Servidor. Criamos um objeto do tipo **java.rmi.registry.Registry,** invocando o método **createRegistry()** da classe **java.rmi.registry.LocateRegistry**. O argumento que passamos nesse método indica a porta TCP na qual o Servidor ficará esperando pelo recebimento de chamadas de clientes RMI. O último passo foi registrar o **Skeleton**, invocando o método **bind()** no objeto do tipo **java.rmi.registry.Registry**, passando como argumento o **Skeleton**.

Com esse código, iniciamos um Servidor RMI que escuta na porta TCP **7777**. Esse valor de porta pode ser qualquer um, embora sejam recomendáveis valores a partir de **1024**. Valores menores que esse costumam pedir permissão de superusuário em sistemas operacionais baseados em Unix. A maior porta permitida no TCP é a **65535**.

A seguir, veremos como implementar um Cliente RMI com capacidade para se comunicar com esse Servidor que criamos.

5.4.4 Implementando um Cliente RMI

Um Cliente RMI é um programa Java com capacidade para enviar solicitações a um Servidor RMI e trabalhar com a resposta recebida. Para criarmos um Cliente com capacidade de se comunicar com o Servidor que acabamos de criar, precisamos apenas da interface **TipoEstabelecimentoServidor**. Podemos implementar esse Cliente no mesmo projeto do Servidor ou em outro. A sugestão é criar um novo projeto, lembrando que ele precisa apenas da interface **TipoEstabelecimentoServidor** no **mesmo pacote** (declaração *package*) do projeto do Servidor. Vejamos como seria o cliente no Código-fonte “Cliente RMI para o “TipoEstabelecimentoService””, na classe que chamamos de **TipoEstabelecimentoCliente**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
// pacote
import java.rmi.NotBoundException;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

import
fiapon.digital.capitulo5.rmi.server.TipoEstabelecimentoServic
e;

public class TipoEstabelecimentoCliente {

     public static void main(String[] args) {
       try {
          Registry registry =
          LocateRegistry.getRegistry("127.0.0.1", 7777);

          TipoEstabelecimentoService stub =
          (TipoEstabelecimentoService) registry
          .lookup("TipoEstabelecimentoService");

          System.out.print("Digite  o  código  de  Tipo  de
estabelecimento: ");
          int codigoPesquisa =
          new Scanner(System.in).nextInt();

          String resultado = stub.pesquisar(codigoPesquisa);

          if (resultado != null) {
               System.out.println(
               "Registro encontrado: "+resultado);
          } else {
               System.out.println(
               "Tipo de estabelecimento não encontrado");
          }

       } catch (RemoteException | NotBoundException e) {
          System.out.println(
          "Erro ao tentar chamada para o servidor");
          e.printStackTrace();
       }
     }

}
```

Código-fonte 5.10 – Cliente RMI para o “TipoEstabelecimentoService”
Fonte: Elaborado pelo autor (2018)

No Cliente, usamos as classes **java.rmi.registry.LocateRegistry** e **java.rmi.registry.Registry** para registrar o Servidor ao qual queremos ter acesso.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No método **getRegistry()**, passamos como argumentos o endereço IP e a porta onde está localizado o servidor RMI com o qual queremos nos comunicar.

Em seguida, solicitamos a criação do **Stub,** usando o método **lookup()** do **java.rmi.registry.Registry**. Como esse método sempre retorna um **Object**, temos que fazer o *cast* para a interface do Servidor, no caso, a **TipoEstabelecimentoService**.

Por fim, basta invocar o método **pesquisar()** da interface **TipoEstabelecimentoService** com o código com o qual queremos realizar a consulta. No exemplo, solicitamos o código via **Scanner** (entrada-padrão do Java).

5.4.5 Fazendo a comunicação entre Servidor e Cliente RMI

Com o Servidor e Cliente criados, agora vamos testar a comunicação entre eles. O primeiro passo é iniciar o Servidor. Para isso, basta executar a classe **TipoEstabelecimentoServidor**. Após iniciada, a saída gerada será apenas a frase "TipoEstabelecimentoService registrado e pronto para aceitar solicitações".

Com o Servidor pronto, basta iniciarmos o Cliente, ou seja, a classe **TipoEstabelecimentoCliente**. Para tornar o exemplo mais interessante, tente executar o Cliente num computador diferente do Servidor, porém na mesma rede. Vamos supor que iniciamos essa classe e indicamos um código de Tipo de Estabelecimento que sabemos existir. Nesse caso, teríamos uma saída como a do Código-fonte "Saída do "TipoEstabelecimentoCliente" informando um código existente":

```
Digite o código de Tipo de estabelecimento: 10

Tipo de estabelecimento encontrado: Parque
```

Código-fonte 5.11 – Saída do "TipoEstabelecimentoCliente" informando um código existente
Fonte: Elaborado pelo autor (2018)

Caso informássemos um código inexistente, a saída seria como a do Código-fonte "Saída do "TipoEstabelecimentoCliente" informando um código inexistente":

```
Digite o código de Tipo de estabelecimento: 50

Tipo de estabelecimento não encontrado
```

Código-fonte 5.12 – Saída do "TipoEstabelecimentoCliente" informando um código inexistente
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Acabamos de ver como criar Cliente e Servidor RMI básicos. Esse procedimento mostra o “caminho das pedras” para a criação de qualquer projeto usando RMI. A seguir, veremos como criar **Sockets** Java, que são usados de forma transparente pelo RMI.

5.5 Sockets

Acabamos de aprender o que é e como funciona o RMI do Java. É possível criar ainda outro jeito de comunicar sistemas em computadores diferentes, que são os **Sockets**. Trata-se de uma tecnologia de mais baixo nível que o RMI, pois, com ela, ocupamos apenas uma porta TCP no servidor socket. A seguir, criaremos uma pequena aplicação Socket Java.

5.5.1 Socket Java que usa um DAO

Como exemplo de Socket, criaremos uma classe que também usará o **TipoEstabelecimentoDAO**. Ele solicitará um código e retornará o nome do registro encontrado ou uma mensagem dizendo que não encontrou registro. Essa classe pode ser criada no mesmo projeto em que foi construído o Servidor RMI. Vejamos o Código-fonte “Classe que usa Socket para pesquisa remota em “TipoEstabelecimentoDAO””:

```
//pacote
import java.io.PrintStream;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class ServidorSocket {

  public static void main(String[] args) throws IOException {
    ServerSocket servidor = new ServerSocket(7777);
    System.out.println("Porta 12345 aberta!");

    Socket cliente = servidor.accept();
    System.out.println(
    "Nova conexão com o cliente "
    + cliente.getInetAddress().getHostAddress());

    PrintWriter saida =
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    new PrintWriter(new PrintStream(
    cliente.getOutputStream()));

    saida.println("Tecle <Enter> para iniciar");
    saida.flush();

    Scanner leitor = new Scanner(cliente.getInputStream());
    while (leitor.hasNextLine()) {
      saida.println();
      saida.print("Pesquisa   por   código   de   Tipo   de
Estabelecimento (0 para sair): ");
      saida.flush();

      int codigoPesquisa = leitor.nextInt();

      if (codigoPesquisa==0) {
           saida.close();
           cliente.close();
      }
      saida.print("Realizando pesquisa, aguarde... ");
      saida.flush();
      TipoEstabelecimento tipoEstabelecimento =
      createDAO().recuperar(codigoPesquisa);

      if (tipoEstabelecimento != null) {
        saida.println(
        "Registro encontrado: "
        + tipoEstabelecimento.getNome());
      } else {
        saida.println("Nenhum registro encontrado!");
      }
      saida.flush();
    }
    saida.close();

    leitor.close();
    servidor.close();
    cliente.close();
  }

  private static TipoEstabelecimentoDAO createDAO() {

    EntityManager em =
    Persistence.createEntityManagerFactory("smartcities")
    .createEntityManager();

    return new TipoEstabelecimentoDAO(em);
  }
}
```

Código-fonte 5.13 – Classe que usa Socket para pesquisa remota em “TipoEstabelecimentoDAO”
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na classe do Código-fonte “Classe que usa Socket para pesquisa remota em “TipoEstabelecimentoDAO””, iniciamos um objeto do tipo **java.net.ServerSocket** indicando no construtor a porta que será ocupada pelo **Socket** (no caso, a **7777**). O construtor dessa classe anuncia uma **IOException**, por isso, repassamos para o método **main()**. Em seguida, criamos um objeto do tipo **java.net.Socket**, o **cliente**. É nesse momento que o **Socket** passa a funcionar e ficar disponível na rede.

O objeto **saida**, do tipo **java.io.PrintWriter**, irá enviar pela rede mensagens de texto para o cliente do Socket. Note que sempre que quisermos que as mensagens sejam enviadas para o cliente, podemos invocar o método **flush()**. Para receber o que o cliente envia, criamos um objeto do tipo **Scanner** a partir do objeto **cliente**.

Ao ser executada, essa classe apenas imprime na saída-padrão “*Porta 7777 aberta!*”.

5.5.2 Acessando o Socket via telnet

Podemos testar o Socket que criamos anteriormente usando apenas um comando do sistema operacional **telnet**, que existe no **Windows**, **Linux** e **MacOS**. No Windows, ele deve ser executado no *prompt de comando*, enquanto no Linux e no MacOS, ele deve ser executado num *terminal bash*. Sugerimos que, se possível, execute os comandos a seguir de um computador diferente, porém na mesma rede, do computador no qual iniciou o **Socket**.

O comando que você deve executar é **telnet 127.0.0.1 7777**. No lugar de **127.0.0.1**, use o IP do computador onde está o **Socket**, caso execute esse comando de outro computador na rede. Ao executar esse comando, você verá um texto como o do Código-fonte “Saída recebida pelo cliente telnet do Socket”:

```
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
Tecle <Enter> para iniciar
```

Código-fonte 5.14 – Saída recebida pelo cliente telnet do Socket
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A partir daí, basta ir digitando um código para pesquisa ou 0 (zero) para desfazer a conexão. Existe uma sequência de consultas e um pedido de saída no Código-fonte “Saída no cliente telnet do Socket após consultas e pedido de saída”:

```
$ telnet 127.0.0.1 7777
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
Tecle <Enter> para iniciar

Pesquisa por código de Tipo de Estabelecimento (0 para sair):
1
Realizando pesquisa, aguarde... Registro encontrado: Escola

Pesquisa por código de Tipo de Estabelecimento (0 para sair):
2
Realizando pesquisa, aguarde... Nenhum registro encontrado!

Pesquisa por código de Tipo de Estabelecimento (0 para sair):
10
Realizando pesquisa, aguarde... Registro encontrado: Parque

Pesquisa por código de Tipo de Estabelecimento (0 para sair):
0

Connection closed by foreign host.
```

Código-fonte 5.15 – Saída no cliente telnet do Socket após consultas e pedido de saída
Fonte: Elaborado pelo autor (2018)

5.5.3 Por que usar Socket, se posso usar RMI?

Talvez você tenha se perguntado: por que usar Sockets, se posso usar RMI, que abstrai mais a comunicação e facilita a programação pelo compartilhamento de interfaces Java? Simples, Sockets, como foi mencionado no início deste tópico, são estruturas de mais baixo nível que RMI, pois só ocupam uma porta diretamente no protocolo TCP. Assim, é possível usar Sockets para disponibilizar o acesso a um projeto Java a dispositivos, como ***Arduino*** e ***Raspberry Pi***, muito usados na prototipação e implementação de projetos **IoT**.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

CALVERT, Kenneth L.; DANAHOO, Michael J. **TCP/IP Sockets in Java:** practical guide for programmers. Burlington: Morgan Kaufmann, 2011.

MOLINARI, Willian. **Desconstruindo a Web.** São Paulo: Casa do Código, 2016.

ORACLE. **Getting started using Java RMI**. Disponível em: <https://docs.oracle.com/javase/7/docs/technotes/guides/rmi/hello/hello-world.html>. Acesso em: 8 jan. 2018.

ORACLE. **Lesson:** all about sockets. Disponível em: <https://docs.oracle.com/javase/tutorial/networking/sockets/index.html>. Acesso em: 8 jan. 2018.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com